
TP 3 — SIMULACIÓN DE MODELOS M/M/1 E INVENTARIO

Tomás Andino

Universidad Tecnológica Nacional — FRRO
Zeballos 1341, S2000, Rosario, Argentina
tomasandino12@gmail.com

Agustín Barroso

Universidad Tecnológica Nacional — FRRO
Zeballos 1341, S2000, Rosario, Argentina
agusbarrosobollero@gmail.com

Facundo Gregoret

Universidad Tecnológica Nacional — FRRO
Zeballos 1341, S2000, Rosario, Argentina
gregoretfacu18@gmail.com

Gerónimo Negri

Universidad Tecnológica Nacional — FRRO
Zeballos 1341, S2000, Rosario, Argentina
geronimonegri@gmail.com

Bruno Schiffo

Universidad Tecnológica Nacional — FRRO
Zeballos 1341, S2000, Rosario, Argentina
schiffobruno2233@gmail.com

1 de julio de 2026

ABSTRACT

El presente trabajo estudia y analiza dos modelos de simulación clásicos: el modelo de colas M/M/1 y el modelo de inventario con política (s, S) . Para cada uno se realizan un mínimo de 10 corridas por experimento, registrando las medidas de rendimiento correspondientes y comparando los resultados obtenidos contra el valor teórico esperado. La comparación se realiza desde tres fuentes: el valor teórico analítico, una implementación propia en Python y un modelo construido en AnyLogic, con el fin de evaluar la convergencia de los resultados simulados al comportamiento ideal del sistema a medida que se incrementa el número de corridas.

Keywords: Simulación · Teoría de Colas · M/M/1 · Inventario · Política (s, S) · AnyLogic · Python

1. Introducción

1.1. Contexto y objetivos del trabajo

El presente trabajo práctico tiene como objetivo estudiar y analizar dos modelos de simulación clásicos: el modelo de colas M/M/1 y el modelo de inventario. Para cada uno se realizan un mínimo de 10 corridas por experimento, registrando las medidas de rendimiento correspondientes y comparando los resultados obtenidos contra el valor teórico esperado, derivado de las fórmulas analíticas de cada modelo.

Esta comparación se realiza desde tres fuentes: el valor teórico, una implementación propia en Python y un modelo construido en AnyLogic, con el fin de evaluar qué tan bien convergen los resultados simulados al comportamiento ideal del sistema a medida que se incrementa el número de corridas.

2. Marco Teórico

2.1. Teoría de Colas — Conceptos Fundamentales

Un **sistema de colas** es aquel en el que un conjunto de clientes (o productos) llega a una estación de servicio, espera en una fila si el servidor está ocupado, recibe atención y luego abandona el sistema [1]. Estos sistemas se encuentran en

numerosas situaciones reales: clientes en un banco, llamadas a un centro telefónico, piezas en una línea de producción, entre otros.

Todo sistema de colas de un solo paso queda completamente caracterizado por cuatro componentes [1]:

- **Población de clientes:** conjunto de todos los posibles clientes que pueden demandar el servicio. Para fines prácticos suele considerarse infinita, salvo que el enunciado indique lo contrario.
- **Proceso de llegada:** describe cómo y con qué frecuencia arriban los clientes. Se caracteriza por la distribución del tiempo entre llegadas sucesivas. En el caso más habitual dicha distribución es **exponencial** con tasa media λ (clientes por unidad de tiempo), lo que genera un *proceso de Poisson*.
- **Proceso de colas:** incluye la capacidad de espera (finita o infinita) y la *disciplina de cola*, es decir, el criterio con que los clientes son seleccionados para ser atendidos. La disciplina más común es **FIFO** (primero en entrar, primero en salir).
- **Proceso de servicio:** define la rapidez con que los clientes son atendidos. Se describe mediante la distribución del tiempo de servicio, cuya media es $1/\mu$, siendo μ la tasa de servicio (clientes por unidad de tiempo).

Notación de Kendall

Para clasificar los modelos de colas se emplea la notación estándar $A/B/c/K/N$, donde cada símbolo representa una característica del sistema:

- A : distribución del tiempo entre llegadas (M = exponencial/Markoviana, D = determinístico, G = general).
- B : distribución del tiempo de servicio (mismas opciones).
- c : número de servidores en paralelo.
- K : capacidad máxima del sistema (cola + servicio). Si se omite se asume $K = \infty$.
- N : tamaño de la población de clientes. Si se omite se asume $N = \infty$.

Medidas de rendimiento

Las **medidas de rendimiento** más utilizadas para evaluar un sistema de colas en estado estable son [1]:

- L : número promedio de clientes en el sistema.
- L_q : número promedio de clientes en la cola (esperando).
- W : tiempo promedio que un cliente invierte en el sistema.
- W_q : tiempo promedio de espera en la cola.
- U : utilización del servidor (fracción de tiempo que está ocupado).
- P_n : probabilidad de que haya exactamente n clientes en el sistema.
- p_w : probabilidad de que un cliente que llega deba esperar.

Estas medidas se relacionan entre sí mediante las **Leyes de Little**:

$$L = \lambda \cdot W \quad (1)$$

$$L_q = \lambda \cdot W_q \quad (2)$$

$$W = W_q + \frac{1}{\mu} \quad (3)$$

donde λ es la tasa de llegada efectiva al sistema.

2.2. Modelo M/M/1 — Definición, Supuestos y Fórmulas

El modelo **M/M/1** es el sistema de colas más sencillo y más estudiado. Consiste en [1]:

1. Una **población de clientes infinita**.
2. Un **proceso de llegada de Poisson** con tasa promedio λ clientes por unidad de tiempo, es decir, los tiempos entre llegadas siguen una distribución exponencial con media $1/\lambda$.
3. Una **única cola** con capacidad ilimitada y disciplina FIFO.
4. Un **único servidor** que atiende según una distribución exponencial con tasa μ clientes por unidad de tiempo.

Condición de estabilidad

Para que el sistema alcance un estado estable es necesario que la tasa de servicio supere a la de llegada:

$$\rho = \frac{\lambda}{\mu} < 1 \quad (4)$$

donde ρ se denomina **intensidad de tráfico** o factor de utilización. Si $\rho \geq 1$, la cola crece indefinidamente y el sistema nunca alcanza el estado estable.

Fórmulas en estado estable

Las medidas de rendimiento del modelo M/M/1 se calculan directamente a partir de ρ [1]:

$$P_0 = 1 - \rho \quad (5)$$

$$P_n = \rho^n \cdot P_0 = (1 - \rho) \rho^n \quad (n = 0, 1, 2, \dots) \quad (6)$$

$$L_q = \frac{\rho^2}{1 - \rho} \quad (7)$$

$$L = \frac{\rho}{1 - \rho} \quad (8)$$

$$W_q = \frac{L_q}{\lambda} \quad (9)$$

$$W = W_q + \frac{1}{\mu} \quad (10)$$

$$p_w = 1 - P_0 = \rho \quad (11)$$

$$U = \rho \quad (12)$$

Estas fórmulas se resumen en la Tabla 1.

Cuadro 1: Fórmulas del modelo M/M/1 en estado estable.

Medida	Fórmula
Intensidad de tráfico	$\rho = \lambda/\mu$
Prob. de sistema vacío	$P_0 = 1 - \rho$
Prob. de n clientes	$P_n = (1 - \rho) \rho^n$
Cientes en cola	$L_q = \rho^2/(1 - \rho)$
Cientes en sistema	$L = \rho/(1 - \rho)$
Tiempo en cola	$W_q = L_q/\lambda$
Tiempo en sistema	$W = W_q + 1/\mu$
Prob. de que un cliente espere	$p_w = \rho$
Utilización del servidor	$U = \rho$

2.3. Cola Finita — Modelo M/M/1/K

En muchos sistemas prácticos el espacio de espera es **limitado**: un estacionamiento, una central telefónica o un buffer de memoria solo pueden alojar un número fijo de clientes esperando a ser atendidos. A este modelo se lo denomina **M/M/1/K** [1].

Convención de notación adoptada. En este trabajo, K representa la **capacidad de la cola de espera** (el número máximo de clientes que pueden esperar, sin contar al que está siendo atendido), y no la capacidad total del sistema. Bajo esta convención, la **capacidad total del sistema** (cola + servidor) es $K + 1$. Esta elección responde directamente a cómo se implementó el modelo tanto en Python como en AnyLogic: en ambos casos, la variable de control (K en Python, K_cola en AnyLogic) determina cuántos clientes pueden aguardar en la cola, mientras que el servidor atiende siempre a uno adicional. El caso $K = 0$ corresponde entonces a un sistema sin espacio de espera (capacidad total = 1), en el cual un cliente solo es admitido si el servidor está libre.

Diferencias respecto al M/M/1

La principal consecuencia de la capacidad finita es que, cuando el sistema ya contiene $K + 1$ clientes (cola llena y servidor ocupado), todo nuevo arribo es **rechazado** y se pierde. Esto implica:

- El sistema **siempre** alcanza el estado estable, incluso cuando $\rho \geq 1$, puesto que la cola no puede crecer indefinidamente.
- La **tasa efectiva de llegada** al sistema es menor que λ , ya que una fracción de clientes es denegada.
- Aparece una nueva medida de rendimiento: la **probabilidad de denegación de servicio** P_b , que representa la probabilidad de que un cliente que llega encuentre el sistema lleno y sea rechazado [1].

Fórmulas del modelo M/M/1/K

Con capacidad total del sistema igual a $K + 1$, las probabilidades de estado son [1]:

$$P_0 = \frac{1 - \rho}{1 - \rho^{K+2}} \quad (\rho \neq 1) \quad (13)$$

$$P_n = \rho^n \cdot P_0 \quad (n = 0, 1, \dots, K + 1) \quad (14)$$

La **probabilidad de denegación de servicio** corresponde al estado en que el sistema está en su capacidad máxima, $n = K + 1$:

$$P_b = P_{K+1} = \rho^{K+1} \cdot P_0 \quad (15)$$

La tasa efectiva de llegada al sistema (descontando los rechazados) es:

$$\lambda_{ef} = \lambda (1 - P_b) \quad (16)$$

El número promedio de clientes en el sistema resulta:

$$L = \frac{\rho}{1 - \rho} - \frac{(K + 2) \rho^{K+2}}{1 - \rho^{K+2}} \quad (17)$$

Las demás medidas se obtienen usando λ_{ef} :

$$W = \frac{L}{\lambda_{ef}} \quad (18)$$

$$W_q = W - \frac{1}{\mu} \quad (19)$$

$$L_q = \lambda_{ef} \cdot W_q \quad (20)$$

Cuando $K \rightarrow \infty$ las fórmulas del M/M/1/K convergen exactamente a las del M/M/1, confirmando que este último es un caso particular del modelo con cola finita. A modo de verificación, con $\rho = 0,75$ y $K = 0$ (capacidad total = 1) se obtiene $P_0 = (1 - \rho)/(1 - \rho^2) = 0,571429$ y $P_b = \rho P_0 = 0,428571$, valor que coincide con el reportado en la Tabla 5 de la Sección 3.

2.4. Modelo de Inventario — Conceptos, Parámetros y Fórmulas

2.4.1. Introducción

Un sistema de inventario es un mecanismo mediante el cual una organización gestiona el stock de uno o más productos con el objetivo de satisfacer la demanda al menor costo posible. En este caso la organización gestiona el stock de un solo producto por fines de simplicidad.

La simulación es especialmente útil en este contexto porque los modelos analíticos exactos existen solo para casos muy simplificados. En la práctica, la demanda es estocástica, los tiempos de reposición son variables y los costos tienen múltiples componentes, lo que hace que la simulación sea la herramienta más adecuada para comparar políticas alternativas.

2.4.2. Política (s, S)

La política de inventario implementada es la denominada política (s, S) o *política de reorden por nivel mínimo y reposición hasta máximo*. Se trata de una política de revisión periódica: al inicio de cada período (cada día) se revisa el nivel de inventario y se decide si ordenar o no. La regla de decisión es:

$$Z = \begin{cases} S - I & \text{si } I < s \\ 0 & \text{si } I \geq s \end{cases} \quad (21)$$

donde I es el nivel de inventario al inicio del período de revisión, s el punto de pedido, S el nivel máximo y Z la cantidad a pedir.

Cuando se emite una orden de Z unidades, estas no están disponibles de inmediato: llegan después de un tiempo de entrega o *lead time*, que en nuestro modelo es una variable aleatoria $L \sim \text{Uniforme}(L_{\min}, L_{\max})$.

2.4.3. Variables de Estado

Se definen las siguientes variables:

Cuadro 2: Variables de estado del modelo de inventario.

Variable	Definición	Dominio
$I(t)$	Nivel de inventario en el tiempo t	$\mathbb{R}$ (puede ser negativo)
$I^+(t) = \max\{I(t), 0\}$	Inventario físico disponible en t	≥ 0
$I^-(t) = \max\{-I(t), 0\}$	Faltante acumulado en t	≥ 0

Cuando $I(t) < 0$, la demanda insatisfecha queda registrada y se satisface en cuanto llegue la próxima orden. Al llegar un pedido, primero se elimina el faltante existente y el excedente (si lo hay) se incorpora al stock positivo.

2.4.4. Proceso de Demanda

La demanda de clientes llega siguiendo un proceso de Poisson con tasa λ clientes por período. Los tiempos entre llegadas son:

$$T_{\text{entre llegadas}} \sim \text{Exp}\left(\frac{1}{\lambda}\right) \quad (22)$$

Cada cliente demanda una cantidad $D \sim \text{Uniforme}(d_{\min}, d_{\max})$ unidades.

2.4.5. Estructura de Costos

El modelo contempla tres componentes de costo:

Costo de orden. Se asume cada vez que se emite una orden:

$$C_{\text{orden}} = K + i \cdot Z \quad (23)$$

donde K es el costo fijo por pedido e i el costo por unidad pedida.

Costo de mantenimiento. Representa el costo de tener unidades físicas en stock:

$$C_{\text{mantenimiento}} = h \cdot \int_0^T I^+(t) dt \quad (24)$$

Costo de faltante. Representa la penalización por demanda insatisfecha:

$$C_{\text{faltante}} = \pi \cdot \int_0^T I^-(t) dt \quad (25)$$

Costo total:

$$C_{\text{total}} = C_{\text{orden}} + C_{\text{mantenimiento}} + C_{\text{faltante}} \quad (26)$$

2.4.6. Promedios por Período

Para comparar políticas con distintos horizontes de simulación, se trabaja con costos promedio por período:

$$\bar{C}_{\text{orden}} = \frac{C_{\text{orden}}}{T}, \quad \bar{C}_{\text{mant}} = \frac{C_{\text{mant}}}{T}, \quad \bar{C}_{\text{falt}} = \frac{C_{\text{falt}}}{T} \quad (27)$$

$$\bar{I}^+ = \frac{1}{T} \sum_{t=1}^T I^+(t), \quad \bar{I}^- = \frac{1}{T} \sum_{t=1}^T I^-(t) \quad (28)$$

de manera que $\bar{C}_{\text{mant}} = h \cdot \bar{I}^+$ y $\bar{C}_{\text{falt}} = \pi \cdot \bar{I}^-$.

2.4.7. Parámetros del Modelo

Cuadro 3: Parámetros del modelo de inventario (s, S) .

Símbolo	Nombre	Descripción
s	Punto de pedido	Nivel mínimo que genera un pedido
S	Nivel máximo	Nivel al que repone el inventario
I_0	Inventario inicial	Nivel de inventario al inicio
λ	Tasa de arribo	Clientes por período (tiempos $\sim \text{Exp}$)
$d_{\text{mín}}, d_{\text{máx}}$	Rango de demanda	Cada cliente demanda $D \sim U(d_{\text{mín}}, d_{\text{máx}})$
$L_{\text{mín}}, L_{\text{máx}}$	Tiempo de entrega	$L \sim U(L_{\text{mín}}, L_{\text{máx}})$
K	Costo fijo de pedido	\$ por pedido, independiente de la cantidad
i	Costo incremental	\$ por unidad pedida
h	Costo de mantenimiento	\$ por unidad por período en stock positivo
π	Costo de faltante	\$ por unidad por período en faltante
T	Tiempo de simulación	Duración total en períodos

3. Modelo M/M/1

3.1. Configuración de los Experimentos

Se fijó una tasa de servicio constante $\mu = 20$ clientes/segundo para todos los experimentos. La tasa de arribo λ se varió tomando el 25 %, 50 %, 75 %, 100 % y 125 % de μ , generando cinco escenarios con $\lambda \in \{5, 10, 15, 20, 25\}$ clientes/segundo, equivalentes a $\rho \in \{0,25, 0,50, 0,75, 1,00, 1,25\}$.

Cada experimento se ejecutó durante $T = 10,000$ segundos de tiempo simulado, con 10 corridas independientes (semillas distintas) por valor de ρ , cumpliendo el mínimo de réplicas exigido por la cátedra.

Para los experimentos de probabilidad $P(n \text{ en cola})$ y de denegación de servicio se utilizó como caso base $\rho = 0,75$ ($\lambda = 15$), por representar un sistema estable pero con ocupación considerable del servidor, adecuado para observar una cola no trivial sin caer en la inestabilidad.

La cola finita se estudió con capacidades $K \in \{0, 2, 5, 10, 50\}$ (entendiendo K como el tamaño de la cola de espera, según se detalla en la Sección 2.3), en las tres fuentes: teórica, Python y AnyLogic. Este último requirió una adaptación puntual del modelo base, ya que el bloque de servicio estándar no ofrece un mecanismo directo de rechazo ante capacidad llena; los detalles de esta implementación se describen en la Sección 3.6.

3.2. Implementación en Python

El simulador M/M/1 se implementó como una simulación de eventos discretos con dos tipos de evento: *arribo* y *fin de servicio*. Los tiempos entre arribos se generan como $\text{Exp}(1/\lambda)$ y los tiempos de servicio como $\text{Exp}(1/\mu)$, ambos mediante el generador de números pseudoaleatorios de Python. El sistema mantiene una cola FIFO de capacidad ilimitada (o limitada a K , en la variante de cola finita) y un único servidor.

Durante la ejecución se acumula, mediante integración numérica del área bajo la curva escalonada, el tiempo ponderado que el sistema pasa con n clientes, lo que permite calcular L y L_q como promedios temporales exactos (no muestrales) al finalizar la corrida. De la misma acumulación se deriva la distribución empírica $P(n)$. La utilización observada ρ_{obs} se calcula como la fracción del tiempo total en que el servidor estuvo ocupado.

Para la variante de cola finita M/M/1/K, se extendió la misma lógica agregando el descarte de arribos cuando el sistema ya contiene K clientes; la probabilidad de denegación P_b se estima como la proporción de arribos rechazados sobre el total de arribos generados.

Cada experimento se repite 10 veces con semillas distintas y se promedian los resultados, reportando los valores agregados en las tablas de las secciones siguientes.

```

1 import random
2
3 def simular_mm1(lam, mu, T, K=None, seed=None):
4     rng = random.Random(seed)
5     t = 0.0
6     n = 0                                # clientes en el sistema
7     ocupado = False
8     area_L = 0.0                         # integral de n(t)
9     area_Lq = 0.0                       # integral de max(n(t)-1, 0)
10    tiempo_ocupado = 0.0
11    arribos = 0
12    rechazados = 0
13
14    t_prox_arribo = rng.expovariate(lam)
15    t_prox_salida = float('inf')
16
17    while t < T:
18        t_evento = min(t_prox_arribo, t_prox_salida)
19        dt = t_evento - t
20
21        area_L += n * dt
22        area_Lq += max(n - 1, 0) * dt
23        if ocupado:
24            tiempo_ocupado += dt
25
26        t = t_evento
27
28        if t_evento == t_prox_arribo:
29            arribos += 1
30            if K is not None and n >= K:
31                rechazados += 1                # cliente denegado
32            else:
33                n += 1
34                if not ocupado:
35                    ocupado = True
36                    t_prox_salida = t + rng.expovariate(mu)
37                t_prox_arribo = t + rng.expovariate(lam)

```

```

38     else:
39         n -= 1
40         if n == 0:
41             ocupado = False
42             t_prox_salida = float('inf')
43         else:
44             t_prox_salida = t + rng.expovariate(mu)
45
46     L = area_L / T
47     Lq = area_Lq / T
48     rho_obs = tiempo_ocupado / T
49     Pb = rechazados / arribos if K is not None else None
50     return L, Lq, rho_obs, Pb

```

Listing 1: Núcleo del simulador de eventos discretos M/M/1 (Python)

3.3. Implementación en AnyLogic

El modelo en AnyLogic se construyó siguiendo el paradigma de eventos discretos mediante la biblioteca de bloques de Procesos. La cadena de bloques utilizada es: source → tmS (marca de tiempo de arribo) → tmS_Q → seize (toma del recurso resourcePool, capacidad 1) → tmE_Q → delay (tiempo de servicio exponencial) → release → tmE → sink.

El bloque source genera arribos con tiempos entre llegadas exponenciales de tasa λ ; el bloque delay aplica un tiempo de servicio exponencial de tasa μ . Dos acumuladores estadísticos, statistics_Queue y statistics_System, registran automáticamente el tiempo medio en cola y en el sistema a partir de las marcas de tiempo tmS, tmS_Q, tmE_Q y tmE. El recurso resourcePool reporta además la utilización porcentual del servidor en tiempo real.

Cada corrida se ejecutó en modo virtual (acelerado) hasta acumular aproximadamente 50.000 clientes procesados, un horizonte equivalente en información estadística al usado en Python. Las Figuras 1 a 5 muestran las capturas del modelo para cada valor de ρ .



Figura 1: AnyLogic — $\lambda = 5$ ($\rho = 0,25$). Utilización del servidor 25 %, $W_q = 0,02$, $W = 0,07$.



Figura 2: AnyLogic — $\lambda = 10$ ($\rho = 0,50$). Utilización del servidor 50 %, $W_q = 0,05$, $W = 0,10$.



Figura 3: AnyLogic — $\lambda = 15$ ($\rho = 0,75$). Utilización del servidor 75 %, $W_q = 0,16$, $W = 0,21$.

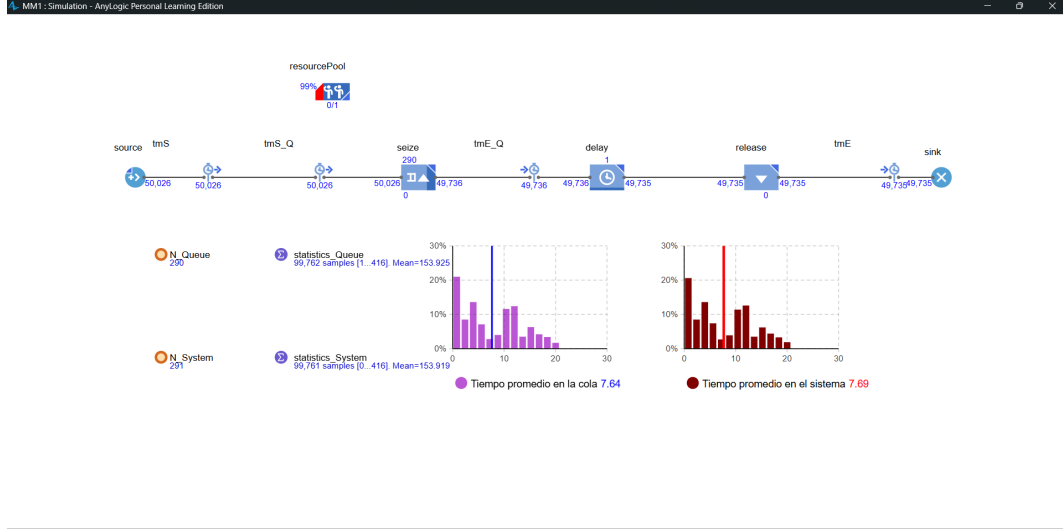


Figura 4: AnyLogic — $\lambda = 20$ ($\rho = 1,00$). Utilización del servidor 99 %, $W_q = 7,64$, $W = 7,69$. El sistema opera al límite de estabilidad y las métricas de tiempo se disparan respecto a los escenarios previos.



Figura 5: AnyLogic — $\lambda = 25$ ($\rho = 1,25$). Utilización del servidor 99 %, $W_q = 206,39$, $W = 206,44$. La cola crece sin control, consistente con un sistema inestable ($\lambda > \mu$).

3.4. Medidas de Rendimiento Finales

La Tabla 4 resume las medidas de rendimiento obtenidas para el caso base $\rho = 0,75$ ($\lambda = 15$), promediadas sobre las 10 corridas, comparadas contra el valor teórico exacto.

Cuadro 4: Medidas de rendimiento — caso base $\rho = 0,75$, $\lambda = 15$, $\mu = 20$ (promedio de 10 corridas).

Métrica	Teórico	Python (media, 10 corridas)	Error relativo
L	3,0000	3,0001	0,003 %
L_q	2,2500	2,2506	0,027 %
W	0,2000	0,2000	0,003 %
W_q	0,1500	0,1500	0,003 %
ρ_{obs}	0,7500	0,7495	0,066 %

Con 10 corridas de 10,000 segundos cada una, el estimador de todas las medidas converge al valor teórico con un error relativo inferior al 0,1 %, lo que confirma la correcta implementación del generador de eventos y de los tiempos exponenciales.

3.5. Medidas de Rendimiento en Relación al Tiempo de Simulación

La Figura 6 muestra la evolución del promedio acumulado de $L(t)$ y $L_q(t)$ a lo largo de la corrida para $\rho = 0,75$. Se observa un período transitorio inicial (aproximadamente hasta $t \approx 1,500$ segundos) donde ambas curvas oscilan por encima del valor teórico, producto de que el sistema arranca vacío ($n = 0$) y necesita varios ciclos de ocupación para “olvidar” esa condición inicial. A partir de ese punto, ambas curvas se estabilizan y convergen asintóticamente a los valores teóricos $L = 3$ y $L_q = 2,25$ (líneas punteadas), evidenciando que $T = 10,000$ segundos es un horizonte suficientemente largo para alcanzar el régimen estacionario en este escenario.

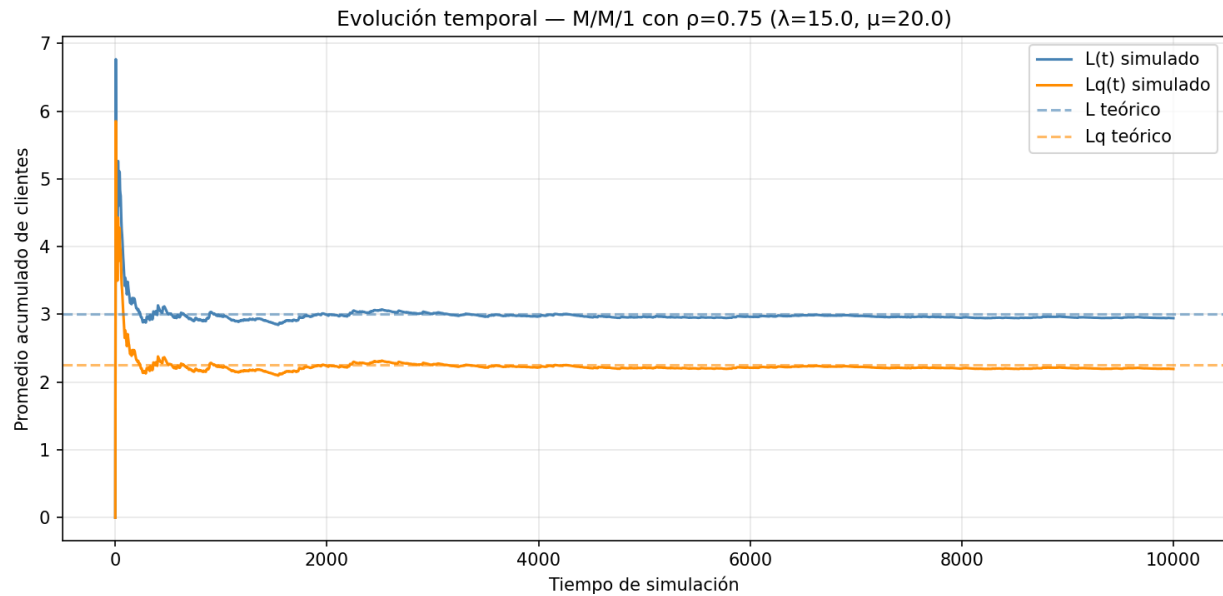


Figura 6: Evolución temporal del promedio acumulado de clientes en sistema $L(t)$ y en cola $L_q(t)$ para $\rho = 0,75$ (Python). Ambas métricas convergen a los valores teóricos tras un breve período transitorio.

3.6. Probabilidad de Denegación de Servicio ($K = 0, 2, 5, 10, 50$)

Para el caso base $\rho = 0,75$ se evaluó la probabilidad de denegación de servicio P_b del modelo M/M/1/K para distintas capacidades de cola K (recordando que, según la convención adoptada en la Sección 2.3, K es el tamaño de la cola de espera y la capacidad total del sistema es $K + 1$). A diferencia del resto de los experimentos de este trabajo, en este caso fue posible construir un modelo de cola finita también en AnyLogic, triangulando las tres fuentes. Los resultados se resumen en la Tabla 5 y se ilustran en las Figuras 7 y 8.

Cuadro 5: Probabilidad de denegación de servicio P_b — M/M/1/K, $\rho = 0,75$: teórico vs. Python vs. AnyLogic.

K	Teórico	Python (10 corridas)	AnyLogic
0	0,428571	0,428915	0,4269
2	0,154286	0,154241	0,1569
5	0,051349	0,050822	0,0522
10	0,010904	0,010859	0,0111
50	≈ 0	0,000000	0,0000

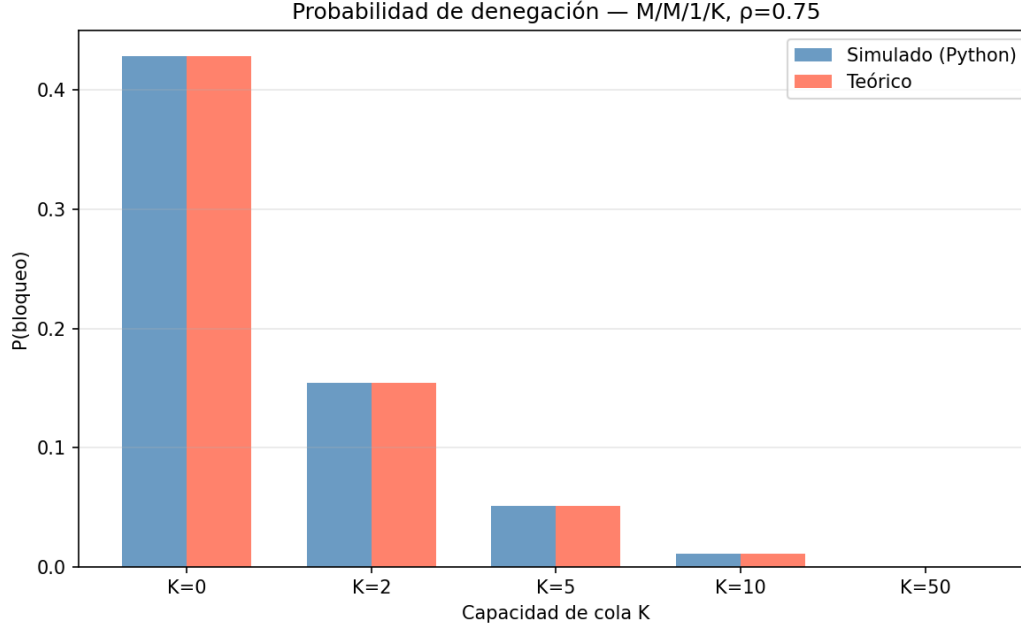


Figura 7: Probabilidad de denegación de servicio P_b en función de la capacidad de cola K , para $\rho = 0,75$: teórico vs. Python.



Figura 8: Modelo M/M/1/K en AnyLogic, $\rho = 0,75$, para $K = 0, 2, 5, 10, 50$ (izquierda a derecha, arriba a abajo). El rechazo de clientes se implementó mediante un bloque `SelectOutput` con la condición `N_System < KCola + 1`, que redirige al cliente hacia un sink alternativo cuando el sistema está en su capacidad máxima. La cola interna del bloque `Seize` se configuró con capacidad máxima (ilimitada) para que el único control de admisión fuera esta condición explícita.

Con $\rho = 0,75$ fijo, P_b decrece de forma aproximadamente geométrica con K (consistente con $P_b = \rho^{K+1}P_0$): al agrandar el buffer, la probabilidad de que un cliente llegue justo cuando el sistema está en su estado más ocupado ($n = K + 1$) cae exponencialmente. Para $K = 50$ la denegación es prácticamente nula en las tres fuentes, resultado esperable ya que con $\rho < 1$ la cola finita se comporta como una cola infinita cuando K es suficientemente grande.

Las tres fuentes coinciden con una diferencia relativa inferior al 2% en todos los valores de K evaluados, lo que constituye una validación cruzada particularmente sólida para este experimento: a diferencia de la variación de ρ (Sección 3.7), acá se cuenta con teórico, Python y AnyLogic simultáneamente. La construcción del modelo de cola finita en AnyLogic no fue inmediata: la implementación inicial, basada en limitar directamente la capacidad de la cola

interna del bloque Seize o en reutilizar bloques RestrictedAreaStart/End de un modelo de ejemplo de la librería, producía errores de ejecución (“An agent was not able to leave the port”), ya que esos mecanismos bloquean al cliente en lugar de redirigirlo. La solución fue reemplazarlos por un bloque SelectOutput con una condición explícita sobre el número de clientes en el sistema ($N_{\text{System}} < K_{\text{cola}} + 1$), dejando la cola interna del Seize sin límite propio para que no compitiera con este control.

3.7. Variación de la Tasa de Arribo (25 % al 125 % de μ)

La Tabla 6 y la Figura 9 resumen el comportamiento de las medidas de rendimiento para los cinco valores de ρ estudiados.

Cuadro 6: Medidas de rendimiento en función de $\rho = \lambda/\mu$ — teórico vs. Python.

ρ	L		W		ρ_{obs}	
	Teórico	Python	Teórico	Python	Teórico	Python
0,25	0,3333	0,3329	0,0667	0,0666	0,2500	0,2492
0,50	1,0000	1,0019	0,1000	0,1002	0,5000	0,5008
0,75	3,0000	3,0001	0,2000	0,2000	0,7500	0,7495
1,00	∞	364,56	∞	18,23	1,0000	0,9970
1,25	∞	24862,49	∞	994,50	1,2500	1,0000

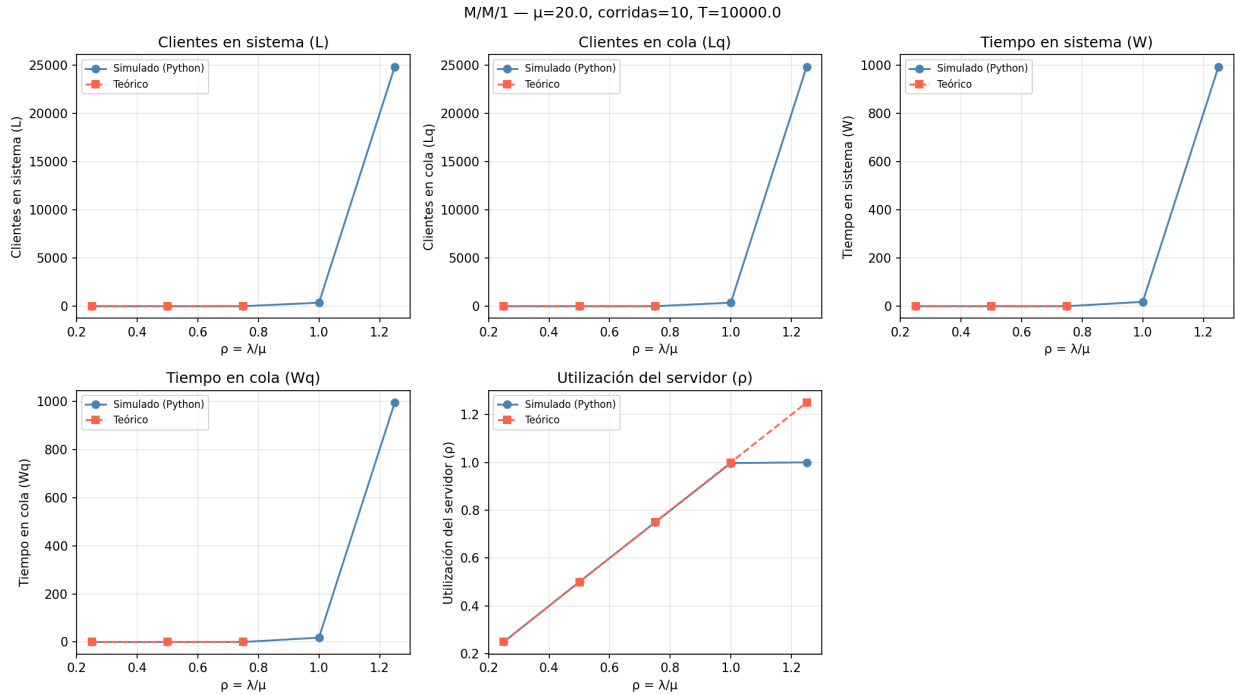


Figura 9: L , L_q , W , W_q y utilización del servidor en función de $\rho = \lambda/\mu$: teórico vs. Python. Para $\rho < 1$ ambas curvas son indistinguibles; a partir de $\rho \geq 1$ el valor teórico diverge a infinito mientras que la simulación reporta valores finitos, acotados por el horizonte de simulación $T = 10,000$ s.

Para $\rho < 1$ el sistema alcanza estado estable dentro del horizonte simulado y las métricas siguen fielmente la curva teórica $L = \rho/(1 - \rho)$, que crece de forma no lineal a medida que $\rho \rightarrow 1$. Para $\rho \geq 1$ el sistema es inestable: el valor teórico es infinito porque la cola crece sin cota a medida que $t \rightarrow \infty$, mientras que la simulación — al estar acotada a $T = 10,000$ s — reporta el tamaño de cola alcanzado hasta ese instante, no un valor de régimen estacionario. Esto explica por qué L pasa de ~ 3 (en $\rho = 0,75$) a 364,56 (en $\rho = 1,00$) y a 24,862,49 (en $\rho = 1,25$): cuanto mayor es el

exceso de λ sobre μ , más rápido crece la cola dentro del mismo tiempo de simulación. La utilización observada ρ_{obs} , en cambio, se satura en 1,0000 para $\rho \geq 1$ porque el servidor está ocupado prácticamente todo el tiempo, algo que sí es consistente con el comportamiento teórico esperado de un sistema saturado.

3.8. Comparación: Valor Teórico vs. Python vs. AnyLogic

La Tabla 7 compara las tres fuentes para el tiempo en cola W_q , el tiempo en sistema W y la utilización del servidor, en cada uno de los cinco escenarios de ρ .

Cuadro 7: Comparación entre valor teórico, Python y AnyLogic — modelo M/M/1.

ρ	W_q			W			Utilización		
	Teo.	Python	AnyLogic	Teo.	Python	AnyLogic	Teo.	Python	AnyLogic
0,25	0,0167	0,0167	0,02	0,0667	0,0666	0,07	25 %	24,92 %	25 %
0,50	0,0500	0,0501	0,05	0,1000	0,1002	0,10	50 %	50,08 %	50 %
0,75	0,1500	0,1500	0,16	0,2000	0,2000	0,21	75 %	74,95 %	75 %
1,00	∞	18,18	7,64	∞	18,23	7,69	100 %	99,70 %	99 %
1,25	∞	994,46	206,39	∞	994,50	206,44	125 %	100 %	99 %

Para $\rho \leq 0,75$ las tres fuentes coinciden con diferencias menores al 5 % en todos los casos, lo que valida cruzadamente tanto la implementación propia en Python como el modelo construido en AnyLogic. Para $\rho \geq 1,00$ las tres coinciden en diverger del comportamiento estable, pero los valores absolutos de Python y AnyLogic difieren considerablemente entre sí ($W_q = 18,18$ vs. 7,64 en $\rho = 1,00$; 994,46 vs. 206,39 en $\rho = 1,25$). Esta discrepancia no invalida ninguna de las dos implementaciones: en un sistema inestable el valor reportado depende fuertemente del horizonte de simulación efectivo, del número de clientes procesados y de la semilla aleatoria particular, ya que no existe un valor de régimen estacionario al cual converger. Ambas implementaciones son, en ese sentido, igualmente “correctas”: están midiendo un sistema que por definición no se estabiliza.

3.9. Gráficas y Análisis

La Figura 10 compara la distribución de probabilidad $P(n)$ de clientes en cola, obtenida analíticamente y por simulación, para el caso base $\rho = 0,75$.

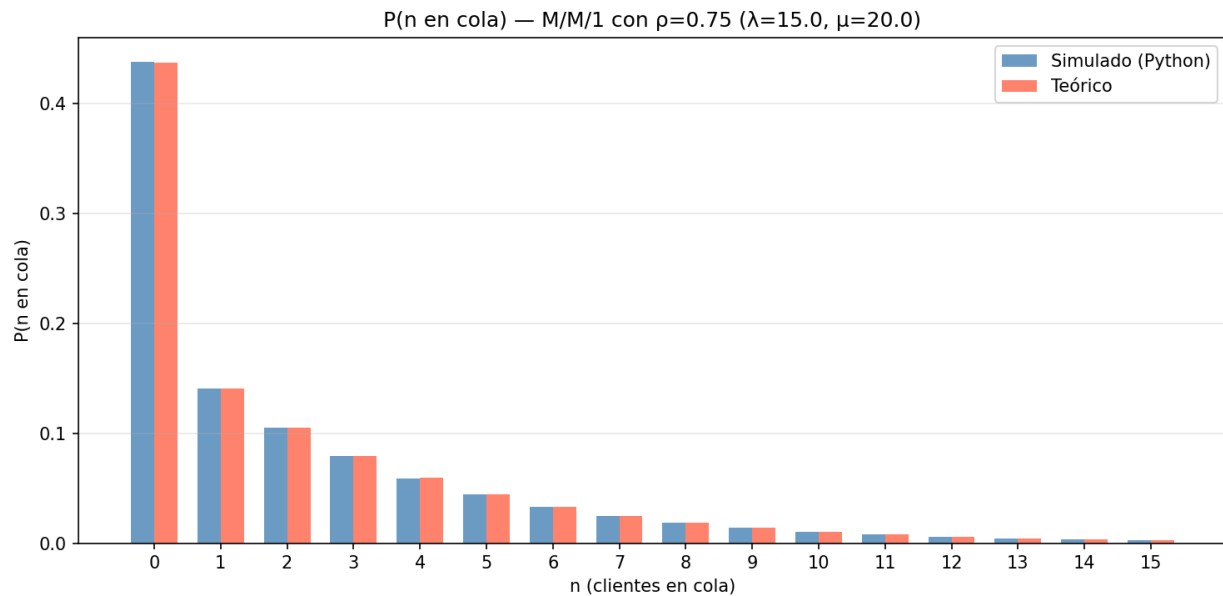


Figura 10: Distribución de probabilidad $P(n \text{ en cola})$ para $\rho = 0,75$ ($\lambda = 15$, $\mu = 20$): teórico vs. Python.

La distribución simulada reproduce con gran fidelidad la forma geométrica decreciente $P_n = (1 - \rho)\rho^n$ prevista por la teoría, incluyendo la cola larga hacia valores altos de n (con probabilidad no despreciable de encontrar más de 10 clientes en cola, consistente con $\rho = 0,75$ relativamente cercano a la inestabilidad).

En síntesis, el conjunto de experimentos muestra tres conclusiones consistentes entre sí:

- **En régimen estable** ($\rho < 1$), las tres fuentes — teórica, Python y AnyLogic — convergen al mismo resultado con errores inferiores al 5 %, lo cual constituye una validación cruzada robusta de ambas implementaciones de simulación.
- **En régimen inestable** ($\rho \geq 1$), la divergencia teórica a infinito se manifiesta en ambos simuladores como valores finitos pero muy elevados y crecientes con ρ , confirmando que lo que se mide no es un estado estacionario sino el crecimiento acumulado de la cola dentro de un horizonte de tiempo finito.
- **La capacidad finita** (K) actúa como mecanismo estabilizador: incluso con $\rho = 0,75$ fijo, aumentar K reduce exponencialmente la probabilidad de denegación, y en el límite $K \rightarrow \infty$ las fórmulas de M/M/1/K convergen a las de M/M/1, tal como se anticipó en el Marco Teórico.

4. Modelo de Inventario

4.1. Selección y Justificación de Parámetros

Los parámetros del modelo fueron seleccionados de manera que los tres componentes de costo (orden, mantenimiento y faltante) resulten observables y comparables entre sí.

Cuadro 8: Parámetros utilizados en la simulación del modelo de inventario.

Parámetro	Valor
Inventario inicial (I_0)	80 (= S)
Punto de reorden (s)	20
Nivel máximo (S)	80
Demanda por cliente	Uniforme(1, 4) uds
Tasa de arribo (λ)	2,5 clientes/día
Lead time	Uniforme(1, 3) días
Costo fijo de orden (K)	\$50 por pedido
Costo incremental de orden (i)	\$3 por unidad pedida
Costo de mantenimiento (h)	\$1 por unidad·día
Costo de faltante (π)	\$10 por unidad·día
Tiempo de simulación (T)	365 días
Corridas	10

4.2. Implementación en Python

El simulador fue desarrollado en Python utilizando las bibliotecas `random`, `numpy` y `matplotlib`.

La simulación avanza día a día durante $T = 365$ períodos. En cada día se ejecutan cuatro pasos en orden fijo: primero se verifica si llega una orden en tránsito y se incorpora al inventario; luego se aplica la política (s, S) y se emite una nueva orden si corresponde; después se genera la demanda del día mediante un proceso de Poisson con tiempos entre llegadas $\text{Exp}(1/\lambda)$; finalmente se acumulan las áreas bajo $I^+(t)$ e $I^-(t)$ para el cálculo posterior de costos.

Al finalizar los 365 días se calculan los tres componentes de costo sobre los acumuladores registrados durante la simulación. Se ejecutan 10 corridas independientes con semillas distintas para garantizar la reproducibilidad. Todos los parámetros son configurables desde la línea de comandos, lo que permite variar valores sin modificar el código.

Simulación Inventario ($s=20$, $S=80$) — 10 corridas $\times$ 365 días

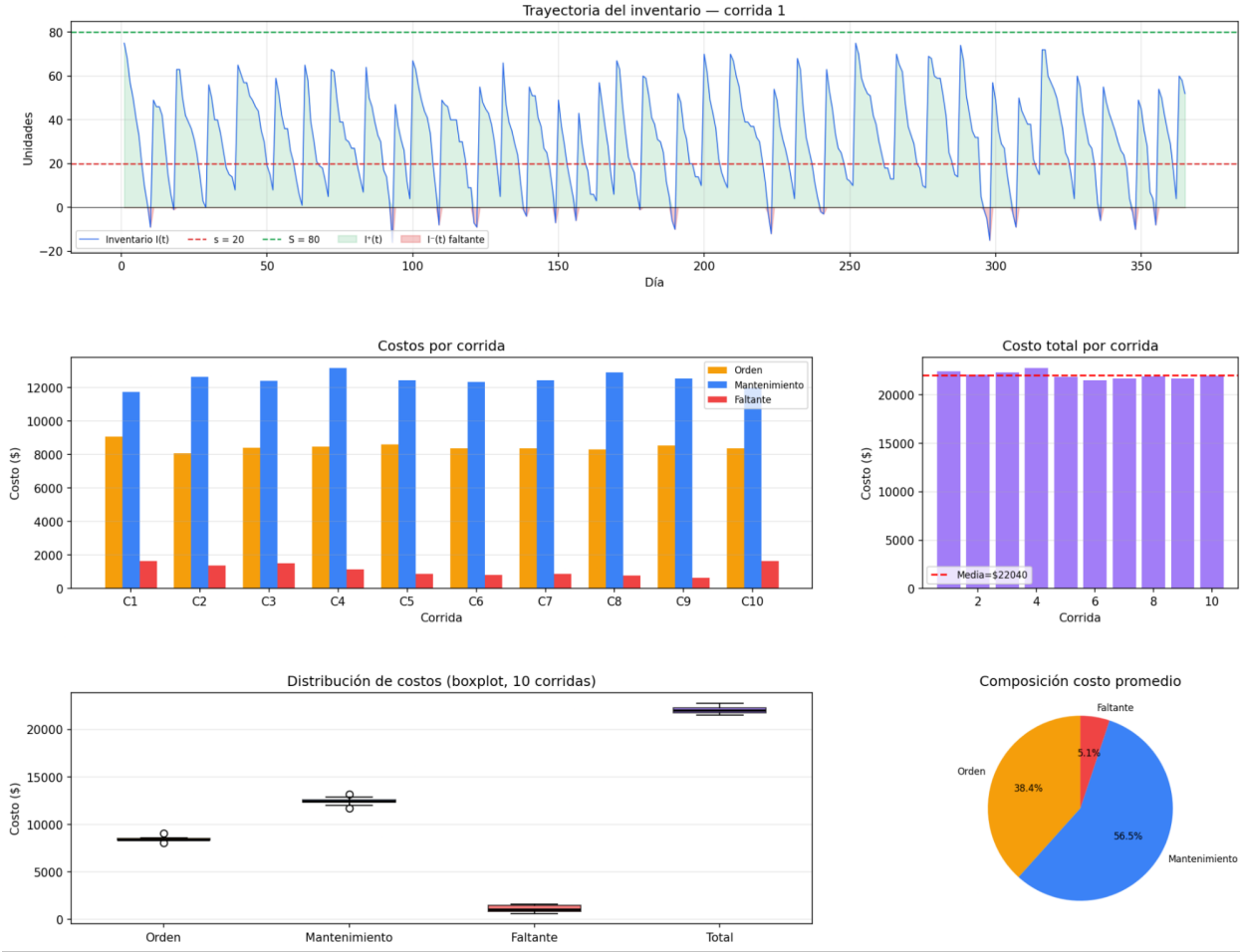


Figura 11: Simulación de inventario ($s = 20$, $S = 80$) — 10 corridas $\times$ 365 días con $\lambda = 2,5$ clientes/día (Python). Se muestra la trayectoria de $I(t)$ en la corrida 1, los costos por corrida, el boxplot de distribución de costos y la composición promedio del costo total.

4.3. Implementación en AnyLogic

El modelo de inventario fue implementado en AnyLogic utilizando el paradigma de simulación de eventos discretos, representando el comportamiento diario del sistema mediante variables de estado y eventos programados.

Parámetros del modelo. Se definieron como parámetros configurables los principales valores del sistema: nivel de reorden (s), nivel objetivo (S), inventario inicial, tasa media de arribos, demanda mínima y máxima por cliente, tiempos mínimo y máximo de entrega, y los cuatro componentes de costo.

Variables de estado. Se utilizaron: inventario actual, cantidad pedida al proveedor, indicador de pedido pendiente, día estimado de llegada del pedido, y los costos acumulados de órdenes, mantenimiento, faltantes y total.

Lógica del modelo. La dinámica se implementó mediante un evento cíclico `DailyUpdate`, ejecutado una vez por día, que realiza en orden: verificación de llegada de pedidos, evaluación de la política (s , S), generación de la demanda diaria (Poisson para número de clientes, Uniforme para cantidad por cliente), actualización del inventario y acumulación de costos.

Controles interactivos. Se incorporaron *sliders* para modificar dinámicamente los parámetros del modelo durante la ejecución, permitiendo analizar escenarios como temporadas de alta demanda, problemas en la cadena de suministro o cambios en el tamaño de los pedidos.

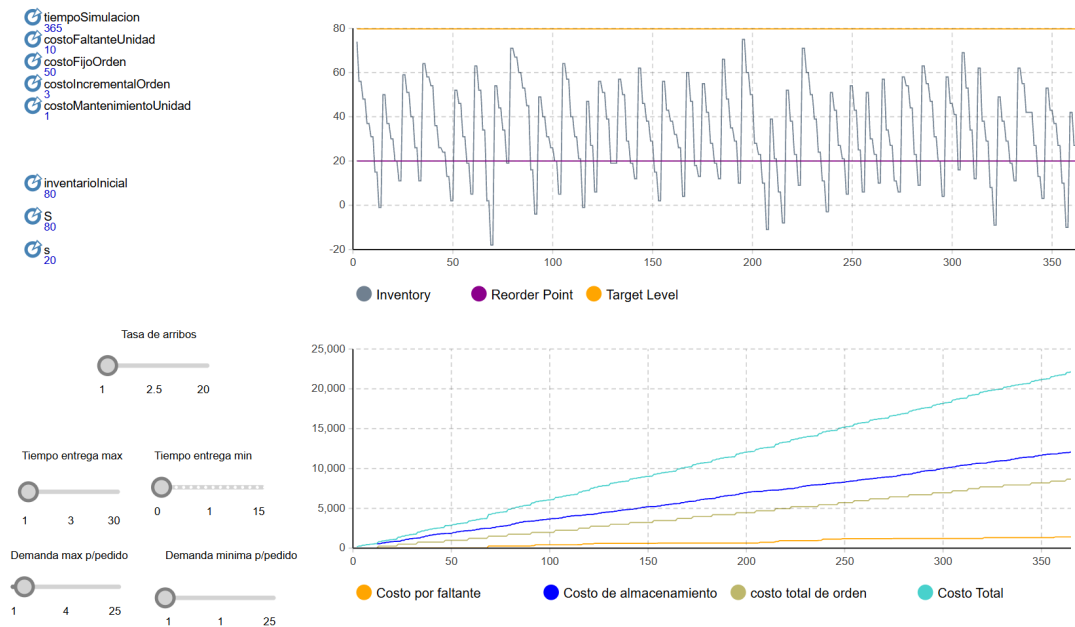


Figura 12: Modelo de inventario en AnyLogic — escenario base ($\lambda = 2,5$ clientes/día). El inventario oscila entre $s = 20$ y $S = 80$ sin episodios de faltante significativos. El costo de almacenamiento domina la composición de costos.

4.3.1. Escenario Base

Durante los primeros 100 días de simulación se mantuvieron los parámetros iniciales definidos para el sistema. El inventario presentó un comportamiento estable, manteniéndose cercano a los niveles definidos por la política (s, S) y sin incurrir en costos significativos por faltantes.

4.3.2. Escenario 1: Incremento de la Demanda

Entre los días 100 y 150 se incrementó la tasa de arribos desde 2,5 hasta 10 clientes por día, simulando una situación de alta demanda o temporada comercial favorable. Se observó una disminución más rápida del inventario y un aumento en la frecuencia de emisión de órdenes. Posteriormente, entre los días 150 y 200, se restablecieron los parámetros originales.

4.3.3. Escenario 2: Problemas en la Cadena de Suministro

Entre los días 200 y 250 se incrementaron los tiempos de entrega, simulando retrasos logísticos. El inventario alcanzó niveles bajos e incluso negativos, produciendo faltante de stock. Entre los días 250 y 300 se restablecieron los valores originales.

4.3.4. Escenario 3: Incremento del Tamaño de los Pedidos

Desde el día 300 hasta el final se aumentó la demanda mínima de 1 a 10 unidades y la máxima de 4 a 25 unidades por cliente, produciendo el mayor impacto sobre el sistema: inventario negativo frecuente y crecimiento acelerado del costo por faltantes.

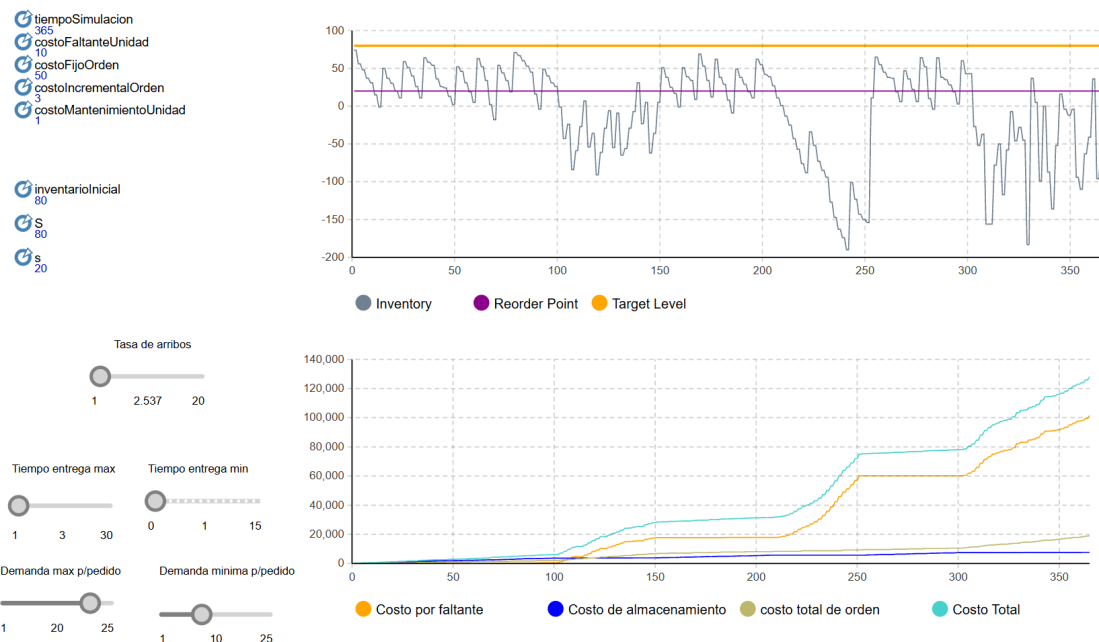


Figura 13: Interfaz de AnyLogic con controles interactivos (*sliders*) mostrando los tres escenarios dinámicos aplicados durante la simulación: incremento de demanda (días 100–150), problemas de suministro (días 200–250) e incremento del tamaño de pedidos (días 300–365).

4.4. Medidas de Rendimiento Finales

Cuadro 9: Costos totales por corrida — modelo de inventario ($s = 20$, $S = 80$).

Corrida	C. Orden (\$)	C. Mantenimiento (\$)	C. Faltante (\$)	C. Total (\$)	Nº Órdenes
1	9071	11736	1640	22447	37
2	8058	12650	1370	22078	33
3	8405	12417	1520	22342	34
4	8470	13161	1140	22771	35
5	8593	12423	880	21896	35
6	8357	12347	810	21514	34
7	8369	12427	890	21686	34
8	8291	12902	760	21953	34
9	8542	12526	640	21708	35
10	8381	11997	1630	22008	34

Cuadro 10: Estadísticas resumidas — modelo de inventario (10 corridas).

Métrica	Media	Desv. Est.	IC 95 % Inf.	IC 95 % Sup.	Mín	Máx
Costo de orden (\$)	8453,70	261,57	8291,65	8615,75	8058	9071
Costo mantenimiento (\$)	12458,60	405,90	12207,17	12710,03	11736	13161
Costo de faltante (\$)	1128,00	382,88	890,52	1365,48	640	1640
Costo total (\$)	22040,30	385,03	21801,66	22278,94	21514	22771
Costo orden/día (\$)	23,16	0,72	22,71	23,60	22,08	24,85
Costo mantenimiento/día (\$)	34,13	1,11	33,44	34,82	32,15	36,06
Costo faltante/día (\$)	3,09	1,05	2,44	3,74	1,75	4,49
Costo total/día (\$)	60,38	1,05	59,73	61,03	58,94	62,39
Número de órdenes	34,50	1,08	33,83	35,17	33	37

4.5. Medidas de Rendimiento en Relación al Tiempo de Simulación

La trayectoria del inventario (corrida 1) muestra que con $s = 20$ y $S = 80$ el nivel $I(t)$ oscila predominantemente en valores positivos, reponiéndose cada ~ 10 días hasta $S = 80$ y descendiendo gradualmente por efecto de la demanda. Los episodios de faltante son escasos y breves, consistente con el bajo costo de faltante observado (\$1128 promedio anual).

La evolución de costos acumulados a lo largo de los 365 días muestra:

- El **costo de mantenimiento** es el de mayor pendiente y el más consistente entre corridas ($CV = 3,3\%$), creciendo de forma casi lineal.
- El **costo de orden** también crece linealmente, reflejando la regularidad del proceso de reposición ($\approx 34,5$ órdenes/año, una cada $\approx 10,6$ días).
- El **costo de faltante** es el único que muestra saltos discretos y mayor variabilidad entre réplicas. Los casos más extremos son las corridas 1 y 10 con \$1640 y \$1630 respectivamente.
- El **costo total** presenta un IC al 95 % estrecho [\$21.801 ; \$22.279], indicando buena estabilidad del estimador con 10 corridas.

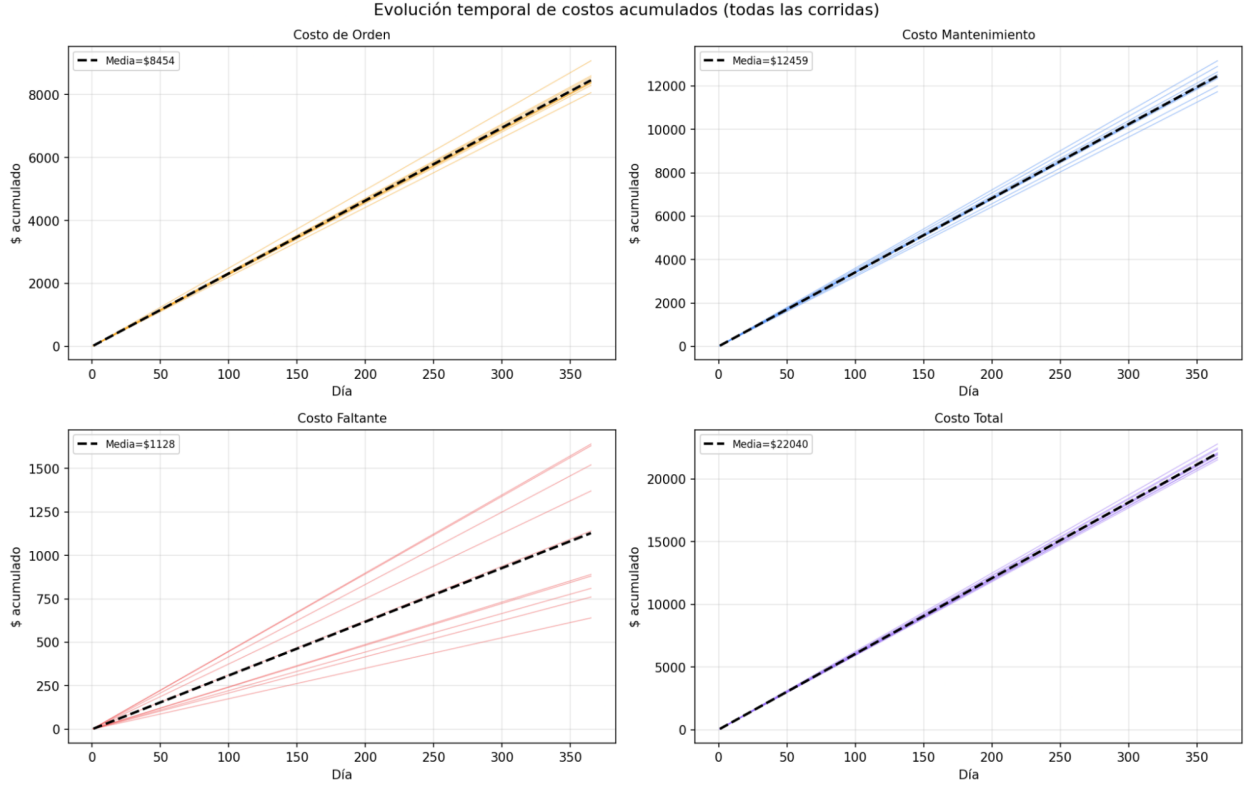


Figura 14: Evolución temporal de costos acumulados para las 10 corridas (Python, $\lambda = 2,5$). El costo de orden y mantenimiento crecen linealmente con baja dispersión entre réplicas; el faltante es el único componente que muestra variabilidad significativa entre corridas.

4.6. Comparación: Valor Teórico vs. Python vs. AnyLogic

4.6.1. Valor Teórico (Aproximación Analítica)

Para el modelo (s, S) con revisión periódica diaria y demanda Poisson, no existe una fórmula cerrada exacta. Se presentan aproximaciones analíticas bajo supuestos simplificadores como referencia.

Demanda diaria esperada:

$$E[D_{\text{día}}] = \lambda \cdot E[D] = 2,5 \times \frac{1+4}{2} = 2,5 \times 2,5 = 6,25 \text{ uds/día} \quad (29)$$

Número esperado de órdenes por año: Dado que el tamaño de orden esperado es $\bar{Z} = S - s = 60$ unidades, el ciclo promedio dura:

$$\bar{T}_{\text{ciclo}} = \frac{\bar{Z}}{E[D_{\text{día}}]} = \frac{60}{6,25} \approx 9,6 \text{ días} \Rightarrow N_{\text{ordenes}} \approx \frac{365}{9,6} \approx 38 \text{ órdenes/año} \quad (30)$$

Costo de orden esperado:

$$C_{\text{orden}}^{\text{teórico}} = N \times (K + i \bar{Z}) = 38 \times (50 + 3 \times 60) = 38 \times 230 = \$8,740/\text{año} \quad (31)$$

Inventario promedio teórico (ciclo triangular perfecto, sin faltante):

$$\bar{I}_{\text{teórico}}^+ = \frac{S + s}{2} = \frac{80 + 20}{2} = 50 \text{ unidades} \quad (32)$$

Costo de mantenimiento esperado:

$$C_{\text{mant}}^{\text{teórico}} = h \cdot \bar{I}_{\text{teórico}}^+ \cdot T = 1 \times 50 \times 365 = \$18,250/\text{año} \quad (33)$$

Costo de faltante: No es posible obtener un valor teórico analítico exacto, ya que requeriría conocer la distribución de la demanda acumulada durante el lead time (suma de variables Poisson y Uniforme sobre $L \sim U(1, 3)$ días).

Costo total esperado (sin faltante):

$$C_{\text{total}}^{\text{teórico}} = 8,740 + 18,250 = \$26,990/\text{año} \Rightarrow \bar{C}_{\text{total}}^{\text{teórico}} = \frac{26,990}{365} \approx \$73,95/\text{día} \quad (34)$$

4.6.2. Tabla Comparativa

Cuadro 11: Comparación entre valor teórico, Python y AnyLogic — modelo de inventario.

Métrica	Teórico (aprox.)	Python (media, 10 corridas)	AnyLogic (media, 10 corridas)
Costo de orden/año (\$)	~8.740	8.453,70	8.585
Costo de mantenimiento/año (\$)	~18.250	12.458,60	12.314
Costo de faltante/año (\$)	N/A	1.128,00	1.308
Costo total/año (\$)	~26.990	22.040,30	22.107
Número de órdenes/año	~38	34,50	35
Costo total/día (\$)	~73,95	60,38	60,57

Los simuladores dan valores de órdenes menores al teórico (~38) porque el inventario raramente cae exactamente a $s = 20$ antes de ordenar — en general cae por debajo, haciendo que $Z = S - I > 60$ unidades, lo que agranda cada orden y alarga el ciclo efectivo. La mayor discrepancia es en el costo de mantenimiento: el modelo analítico sobreestima en casi un 50 % porque el supuesto de ciclo triangular perfecto no captura la variabilidad real de la demanda Poisson, que mantiene el inventario en niveles más bajos durante más tiempo.

4.7. Gráficas y Análisis

4.7.1. Análisis de Sensibilidad

Se identifican las siguientes formas de control del sistema:

Cuadro 12: Análisis de sensibilidad del modelo de inventario.

Modificación	Efecto esperado
Aumentar s	Realizar el pedido antes: menos faltante pero mayor costo de mantenimiento y de pedido.
Aumentar S	Pedidos más grandes y menos frecuentes: menos costo fijo pero más mantenimiento.
Aumentar λ	Más demanda diaria: más faltante, ciclos más cortos.
Aumentar π	No cambia el comportamiento físico, sí el costo total al penalizar más el faltante.
Reducir $L_{\text{máx}}$	Menos exposición durante el tránsito: menos faltante.

Escenario $\lambda = 10$ clientes/día. Al incrementar λ de 2,5 a 10 clientes/día, la demanda diaria esperada pasa de 6,25 a 25 unidades/día, superando ampliamente la capacidad de reposición del sistema. El inventario $I(t)$ cruza el cero con alta frecuencia y pasa la mayor parte del año en valores negativos, llegando a episodios de faltante de hasta -100 unidades. El costo de faltante representa el 71,5 % del total y el costo total medio asciende a \$123.322, más de cinco veces el obtenido con $\lambda = 2,5$. Con este nivel de demanda, la política ($s = 20$, $S = 80$) resulta inadecuada: $S = 80$ representa apenas 3,2 días de demanda esperada.

Simulación Inventario ($s=20$, $S=80$) — 10 corridas x 365 días

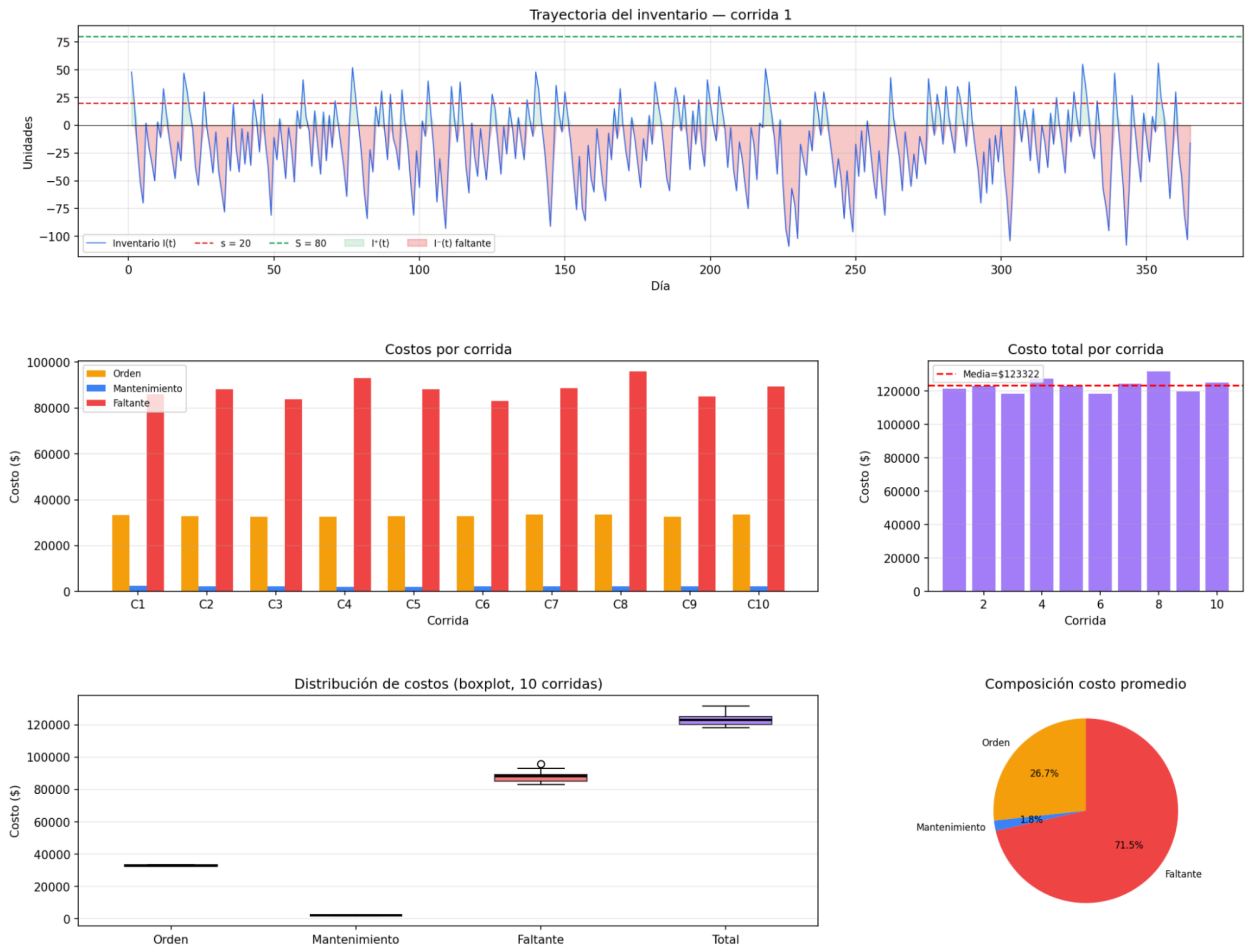


Figura 15: Python con $\lambda = 10$ clientes/día: el inventario pasa la mayor parte del año en valores negativos. El faltante representa el 71,5 % del costo total medio (\$123.322), más de cinco veces el escenario base.



Figura 16: AnyLogic con $\lambda = 10$ clientes/día: comportamiento consistente con Python. El faltante domina los costos y el inventario cruza el cero con alta frecuencia durante todo el año.

Escenario $\lambda = 1$ cliente/día. Al reducir λ a 1 cliente/día, la demanda diaria esperada pasa de 6,25 a 2,5 unidades/día. El inventario nunca llega a cero: desciende lentamente desde $S = 80$ manteniéndose por encima de $s = 20$ durante períodos prolongados. El mantenimiento representa el 82,3 % del total y el costo de faltante es 0,0 %. El sistema es muy estable con mínima variabilidad entre corridas.

Simulación Inventario ($s=20$, $S=80$) — 10 corridas $\times$ 365 días

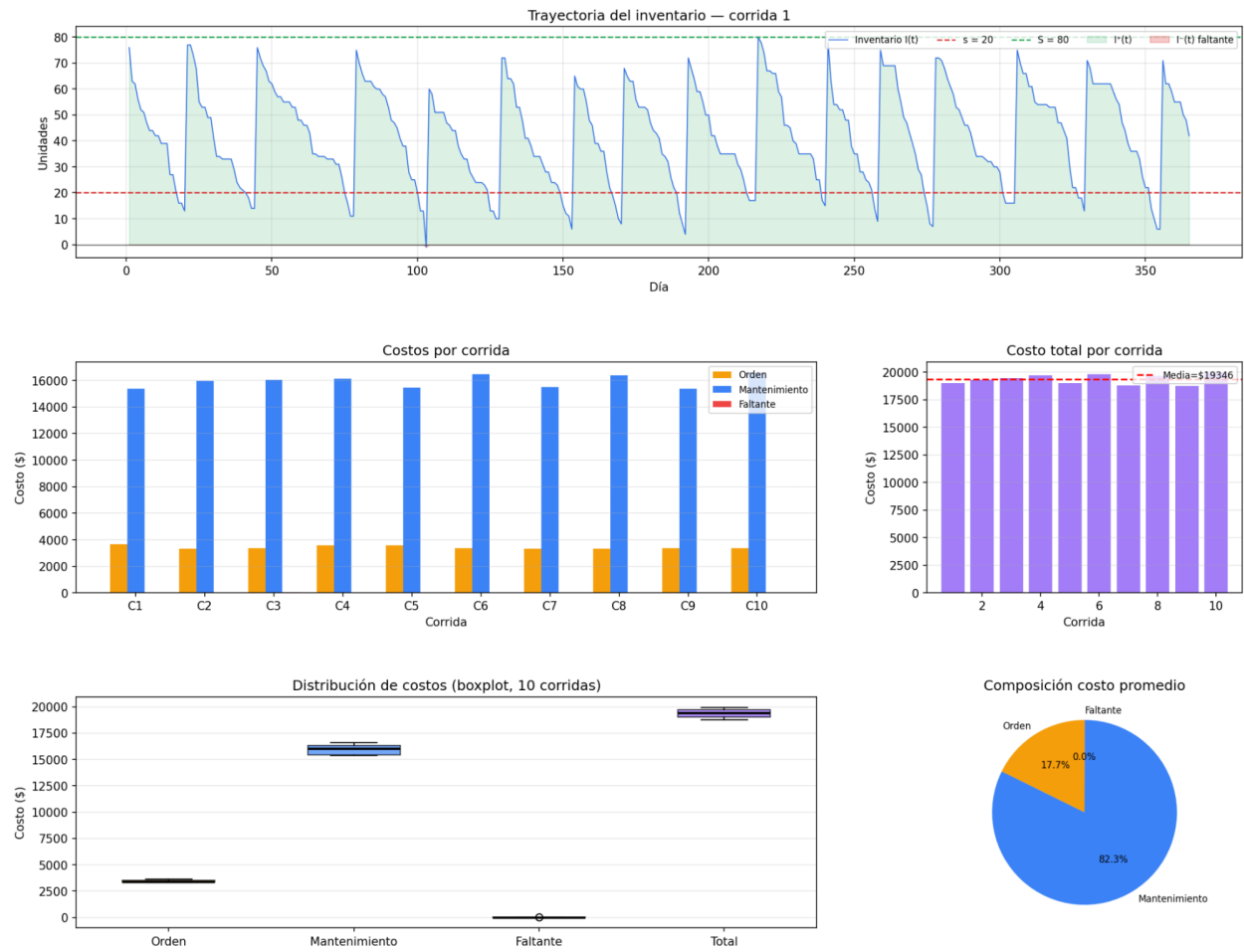


Figura 17: Python con $\lambda = 1$ cliente/día: el inventario nunca llega a cero. El mantenimiento representa el 82,3 % del costo total medio (\$19.346). El faltante es prácticamente inexistente y el sistema presenta baja variabilidad entre corridas.

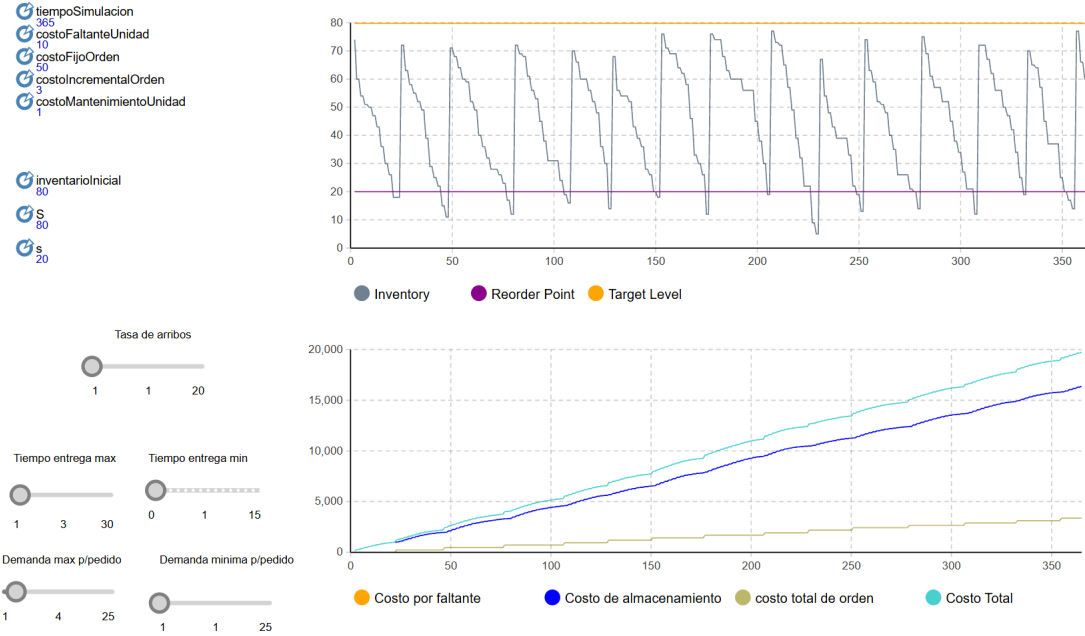


Figura 18: AnyLogic con $\lambda = 1$ cliente/día: ciclos de reposición largos y regulares, sin faltante. El costo de almacenamiento domina abrumadoramente la composición de costos.

5. Conclusiones

El presente trabajo permitió estudiar dos modelos de simulación de naturaleza distinta — un sistema de colas M/M/1 (proceso markoviano puro, con solución analítica cerrada) y un sistema de inventario con política (s, S) (sin solución analítica exacta) — y comparar en ambos casos los resultados obtenidos por vía teórica, por una implementación propia en Python y por un modelo construido en AnyLogic.

En el modelo M/M/1, la coincidencia entre las tres fuentes en régimen estable ($\rho < 1$), con errores relativos inferiores al 5 %, valida tanto la corrección de la implementación del generador de eventos discretos en Python como la configuración del modelo en AnyLogic. El experimento de cola finita M/M/1/K reforzó esta validación de manera particularmente completa: a diferencia de otros experimentos de este trabajo, aquí fue posible triangular las tres fuentes simultáneamente (teórico, Python y AnyLogic), mostrando que la probabilidad de denegación de servicio simulada reproduce con alta precisión la fórmula geométrica teórica $P_b = \rho^{K+1} P_0$ para los cinco valores de K evaluados, con diferencias relativas inferiores al 2 % en todos los casos. Lograr esta triangulación requirió además un ajuste puntual sobre el modelo base de AnyLogic, reemplazando el mecanismo de bloqueo implícito de los bloques de servicio estándar por una condición explícita de rechazo, lo cual ilustra que — a diferencia de las fórmulas cerradas del modelo teórico — las herramientas de simulación de propósito general muchas veces requieren adaptaciones deliberadas para capturar correctamente un supuesto particular del sistema en estudio. El caso $\rho \geq 1$ resultó particularmente instructivo: permitió visualizar de forma concreta la diferencia entre un valor teórico que diverge por definición y una simulación de horizonte finito, que en cambio reporta el estado transitorio de una cola en crecimiento indefinido, sin alcanzar nunca el régimen estacionario que las fórmulas asumen.

En el modelo de inventario, la ausencia de una solución analítica exacta obligó a construir una aproximación simplificada (ciclo triangular determinístico), que resultó razonablemente precisa para el costo de orden pero sobreestimó en cerca de un 50 % el costo de mantenimiento, al no capturar la variabilidad introducida por la demanda de Poisson y el lead time aleatorio. En este caso, Python y AnyLogic — ambos capaces de modelar la estocasticidad completa del sistema — coincidieron entre sí con una diferencia inferior al 3 % en el costo total anual, lo que sugiere que la discrepancia observada respecto del valor teórico proviene del propio supuesto simplificador y no de un error de implementación. El análisis de sensibilidad, tanto mediante variación paramétrica en Python como mediante los escenarios interactivos de AnyLogic, permitió además identificar el impacto diferencial de cada parámetro: aumentos en la tasa de arribo o en el tamaño de la demanda por cliente desplazan rápidamente el sistema hacia un régimen dominado por el costo de

faltante, mientras que la política base ($s = 20$, $S = 80$) resulta adecuada únicamente para el escenario de demanda originalmente calibrado.

En conjunto, ambos casos de estudio ilustran el valor de la simulación como herramienta complementaria — y en muchos casos necesaria — al análisis teórico: en el modelo M/M/1 permitió observar comportamientos (como el estado transitorio y la divergencia práctica en sistemas inestables) que la fórmula cerrada no expone directamente, mientras que en el modelo de inventario constituyó la única vía disponible para obtener estimaciones confiables de costos ante la ausencia de una solución analítica exacta. La utilización de dos plataformas de simulación independientes (una implementación propia en Python y un modelo en AnyLogic) resultó clave para distinguir entre errores de implementación y comportamientos genuinos del sistema, incrementando la confiabilidad de las conclusiones alcanzadas en ambos modelos.

6. Referencias

Referencias

- [1] Hillier, F.S., Lieberman, G.J. *Introducción a la Investigación de Operaciones*, 9na ed. McGraw-Hill, 2010.
- [2] Banks, J., Carson, J., Nelson, B., Nicol, D. *Discrete-Event System Simulation*, 5th ed. Prentice Hall, 2010.
- [3] AnyLogic. *AnyLogic Documentation*. Disponible en: <https://anylogic.help/>
- [4] Python Software Foundation. *random* — Generate pseudo-random numbers. Disponible en: <https://docs.python.org/3/library/random.html>
- [5] Matplotlib Development Team. *Matplotlib Documentation*. Disponible en: <https://matplotlib.org/stable/index.html>
- [6] Cátedra de Simulación. Apuntes teóricos de la materia. Universidad Tecnológica Nacional — FRRO, 2026.